Chapter 1

Evaluation

1.0.1 Purpose

The purpose of this evaluation is to empirically assess the effectiveness, reliability, and practical utility of Houdini as a framework for detecting security issues, misconfigurations, and enforcement inconsistencies in containerized environments.

1.0.2 Requirements / Goals:

- **Reproducibility:** Houdini should produce consistent and repeatable results when run across different machines and environments. Identical configurations and test inputs should yield the same outcomes.
- **Scalability:** Houdini should support scalable execution, allowing users to easily configure, queue, and execute large batches of tests/tricks across multiple container configurations or versions. The framework should be designed to accept input declaratively and automate the full evaluation without requiring manual intervention.
- **Performance:** Houdini should execute tests efficiently, minimizing runtime overhead while maintaining accuracy. The framework's performance should

allow it to be integrated into larger workflows, such as continuous integration pipelines, without introducing significant delays.

- **Security and Host Isolation:** Houdini must isolate tests from the host system to ensure safety during the execution of potentially dangerous tricks. By running tests inside a virtual machine, Houdini should prevent container escapes or exploits from impacting the host. It should maintain a secure boundary between the test environment and the underlying system.

- **Accuracy of Detection** Houdini should correctly distinguish between successful and failed security isolation. Test results should reflect the actual runtime behavior of the container under evaluation without false positives or negatives.

- **Flexibility Across Configurations** The framework should support a wide range of container settings, including different Linux capabilities, namespaces, AppArmor profiles, seccomp filters, and Docker flags. It should also accommodate various versions of Docker, runc, crun, and the Linux kernel.

- **Transparency and Debuggability** Test runs should be transparent and easy to interpret. Logs, intermediate outputs, and error messages should be informative enough to help users understand why a trick passed or failed.

- **Detect Configuration-Behavior Inconsistencies:** Evaluate whether containers behave as expected under different security configurations by empirically validating real-world runtime behavior. This helps identify mismatches between intended policy and actual enforcement.

- **Assess Cross-Version Consistency and Enforcement Drift:** Determine whether the same container configuration yields consistent behavior across different versions of Docker, container runtimes (e.g., runc, crun), and Linux kernels.

1.0.3 Methodology / How we will do that:

To evaluate the effectiveness and design properties of the Houdini framework, we conduct a series of controlled experiments targeting key aspects of its architecture and operation. Each experiment is designed to test a specific goal of the framework, including reproducibility, scalability, performance, and security isolation.

We structure our methodology as follows:

1. **Reproducibility Across Machines**

   Run the same trick and configuration on two different machines (e.g., different CPU architectures or hardware specs) and verify that the output is identical.

2. **Reproducibility Across Time with Version Pinning**

   Re-run a pinned Docker, kernel, and runc configuration at a later time to ensure consistent results, demonstrating Houdini's ability to reproduce experiments reliably.

3. **Scalability via Bulk Execution**

   Execute a large batch of tricks (e.g., 32) across multiple configurations in one session. Measure runtime, memory usage, and failure rates to assess Houdini's batch execution capabilities.

4. **Performance Benchmarking**

   Measure the end-to-end time it takes Houdini to execute a single trick, including VM boot, container launch, and result collection. Compare this to a manually executed baseline outside Houdini.

5. **Host Isolation Validation**

   Run a deliberately malicious trick (e.g., attempting to write to host directories) and verify that the host remains secure, demonstrating that Houdini's use of VMs effectively isolates test environments.

6. **Fault Tolerance and Crash Recovery**

   Introduce failing or crashing tricks (e.g., segmentation faults, infinite loops) and confirm that Houdini logs the failure and continues running subsequent tests without interruption.

7. **Version Variability Support**

   Create a test matrix across different combinations of Docker, runc, and kernel versions. Confirm that Houdini successfully builds and runs tricks across all variations.

8. **Extensibility Test**

   Develop a new custom trick (e.g., testing for a capability or syscall) and integrate it into Houdini. Measure the effort and steps needed to add and run it without modifying the core logic.

9. **CI Integration Test**

   Integrate Houdini into a CI/CD system such as GitHub Actions or GitLab CI. Trigger it on container configuration changes and assess its stability and reporting in an automated pipeline.

1.0.4 What Did You Find Out?

TODO when I actually run the experiments, but for the last two goals I have some results:

- The goal **"Detect Configuration-Behavior Inconsistencies"** is directly supported by **Case Studies 2, 3, and 4**, where the same configurations sometimes led to unexpected or insecure behavior (e.g., host interaction without capabilities, port binding despite restrictions, and ptrace behavior differing by scope).

- The goal **"Assess Cross-Version Consistency and Enforcement Drift"** is supported by **Case Studies 1, 3, and 4**, where the same trick or config yielded different results across versions of Docker, runc, and Linux distributions.

- **Case Study 1 (CVE-21626):** Discovered that older versions of `runc` (e.g., rc1–rc4), not listed in the official CVE disclosure, were also vulnerable. This expanded the known scope of the vulnerability and revealed version-scoping inconsistencies.

- **Case Study 2 (Host Process Manipulation):** Identified the minimum set of privileges required for a container to successfully interact with host processes. Demonstrated that certain combinations of namespace sharing and UID matching can bypass expected isolation controls, even with reduced capabilities.

- **Case Study 3 (Docker Privileged Port Binding):** Found that containers could bind to privileged ports under newer Docker versions despite appearing unprivileged. This revealed misconfigurations and inconsistencies between intended policy and actual runtime behavior.

- **Case Study 4 (ptrace Enforcement Across Linux Distributions):** Empirically validated that ptrace enforcement behavior varies across Linux distributions due to differences in kernel versions and default Yama settings. Exposed enforcement drift[1] and platform-dependent inconsistencies in confinement guarantees.

[1] Drift refers to changes in security behavior over time due to image updates, runtime automation, or infrastructure modifications.